4160 – Distributed and Parallel Computing

P3 – Hardware software co-design

CUHK

Hardware software co-design

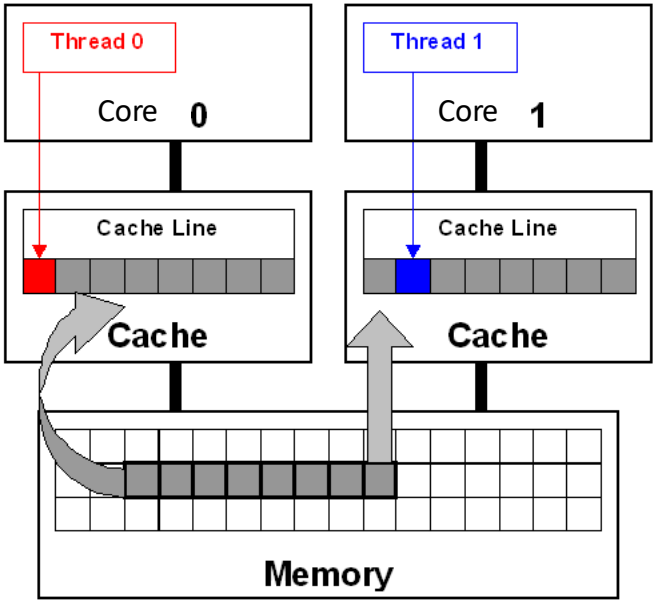
- How to improve the performance of your program?
- Algorithm complexity is still the primary factor. But after that
  - Cacheline ping-pong?
  - Thread-reuse?
  - Bandwidth bound? (Topic P5)

For pioneering contributions to numerical algorithms and libraries that enabled high performance computational software to keep pace with exponential hardware improvements for over four decades. Turing Award 2021 Jack Dongarra

Cache coherence and False sharing

- int sum_local[2]; // small enough to fit in the **same cache line** (64 bytes)*
  - Thread 0 updates sum_local[0];
  - Thread 1 updates sum_local[1];
- **False sharing** causes cache-line **ping-pong**!

```
01 double sum=0.0, sum_local[NUM_THREADS];
02 #pragma omp parallel num_threads(NUM_THREADS)
03 {
04  int me = omp_get_thread_num();
05  sum_local[me] = 0.0;
06
07  #pragma omp for
08  for (i = 0; i < N; i++)
09  sum_local[me] += x[i] * y[i];
10
11  #pragma omp atomic
12  sum += sum_local[me];
13 }
```

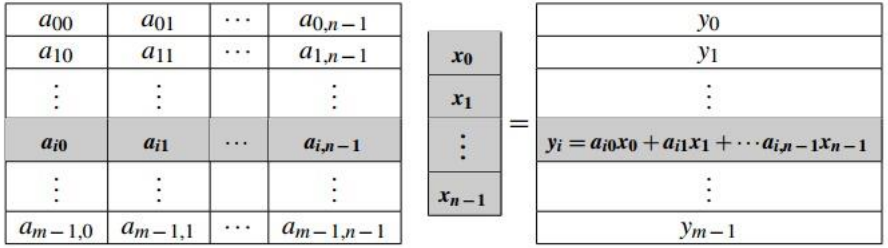


https://software.intel.com/en-us/articles/avoiding-and-identifying-false-sharing-among-threads

*E.g., CPU cache lines align with 64 bytes memory boundaries. But malloc may allocate a[0] at the end of a cacheline, which makes a[1] starts at a new cache line, unless you *memalign* it

Parallel Matrix Vector Multiplication

- y = Ax
- A = m x n matrix
- x = n x 1 vector



Pacheco's book

```
/* For each row of A */
for (i = 0; i < m; i++) {
   y[i] = 0.0;
   /* For each element of the row and each element of x */
   for (j = 0; j < n; j++)
      y[i] += A[i][j]* x[j];
}
```

Parallelizing the outer for loop

```
#pragma omp parallel for private(i,j) shared(A,x,y,m,n)
for (i = 0; i < m; i++) {
    y[i] = 0.0;
    /* For each element of the row and each element of x */
    for (j = 0; j < n; j++)
       y[i] += A[i][j]* x[j];
}
```

Parallel Matrix Vector Multiplication

- Efficiency = Speedup / # of threads
  - Ideal: E = 1
- All 3 cases involve
  - 64 million multiplications and additions
  - ➔ but different running time and efficiency?

Table 4.5 Run-Times and Efficiencies of Matrix-Vector Multiplication (times are in seconds)

	Matrix Dimension					
	$8{,}000{,}000 \times 8$		8000×8000		$8 \times 8{,}000{,}000$	
Threads	Time	Eff.	Time	Eff.	Time	Eff.
1	0.393	1.000	0.345	1.000	0.441	1.000
2	0.217	0.906	0.188	0.918	0.300	0.735
4	0.139	0.707	0.115	0.750	0.388	0.290

$$\begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \vdots & \vdots & & \vdots \\ a_{i0} & a_{i1} & \cdots & a_{i,n-1} \\ \vdots & \vdots & & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots a_{i,n-1}x_{n-1} \\ \vdots \\ y_{m-1} \end{bmatrix}$$

Parallelizing the outer for loop
E.g., 4 threads, then each thread is responsible for
//for 8mx8 * 8*1 vector => y is 8m x 1
y[1 .. 2m]
y[2m+1 .. 4m]
y[4m+1 .. 6m]
y[6m+1 .. 8m]

Pacheco's book

```
#pragma omp parallel for private(i,j) shared(A,m,n,   x,y)
for (i = 0; i < m; i++) {
    y[i] = 0.0;
    /* For each element of the row and each element of x */
    for (j = 0; j < n; j++)
       y[i] += A[i][j]* x[j];
}
```

Parallel Matrix Vector Multiplication

- Why 8x8m is the worst under 4-threads?
- 8m x 8 input matrix A
  - Resulting vector y has 8m components (tall)
  - Each thread is assigned with 2m components
    - Each works on its sub-2m y components **independently**
- 8k x 8k input matrix A
  - Resulting vector y has 8k components
  - Each thread is assigned with 2k components
- 8 x 8m input matrix A [worst]
  - Resulting vector y has 8 components
  - **y fits in a single cache-line**
  - Threads share the same cache line of y
    - **false-sharing!**
  - **Ping-pong through memory!**

Table 4.5 Run-Times and Efficiencies of Matrix-Vector Multiplication (times are in seconds)

	Matrix Dimension					
	$8{,}000{,}000 \times 8$		8000×8000		$8 \times 8{,}000{,}000$	
Threads	Time	Eff.	Time	Eff.	Time	Eff.
1	0.393	1.000	0.345	1.000	0.441	1.000
2	0.217	0.906	0.188	0.918	0.300	0.735
4	0.139	0.707	0.115	0.750	0.388	0.290

$$\begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \vdots & \vdots & & \vdots \\ a_{i0} & a_{i1} & \cdots & a_{i,n-1} \\ \vdots & \vdots & & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots a_{i,n-1}x_{n-1} \\ \vdots \\ y_{m-1} \end{bmatrix}$$

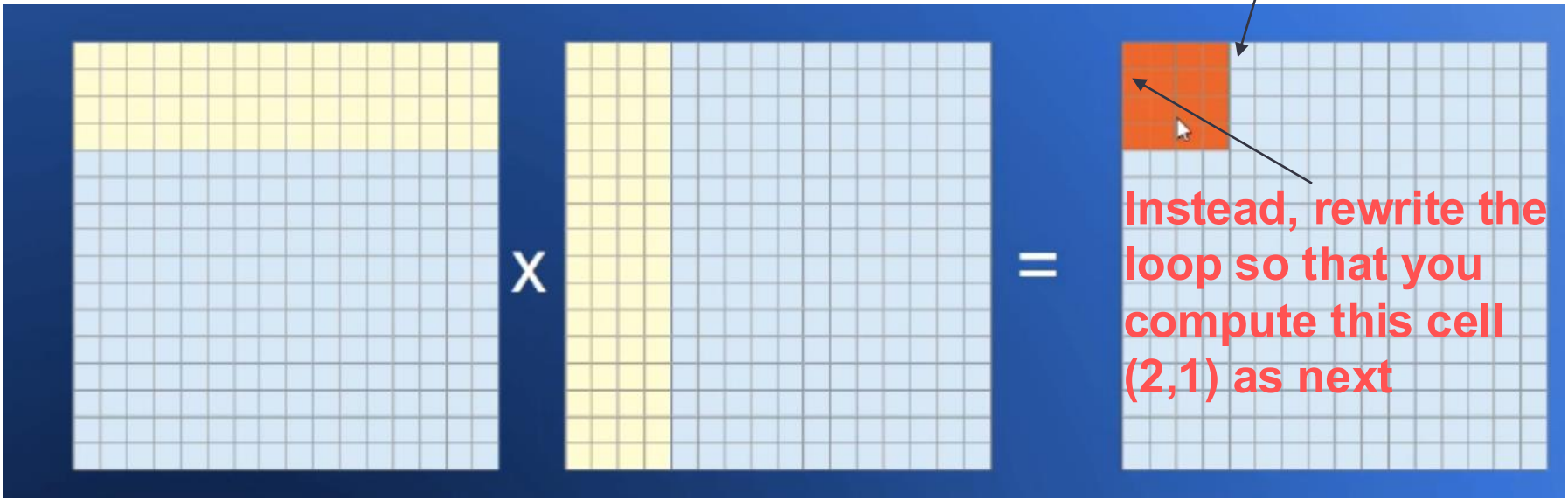
Pacheco's book

```
#pragma omp parallel for private(i,j) shared(A,x,y,m,n)
for (i = 0; i < m; i++) {
   y[i] = 0.0;
   /* For each element of the row and each element of x */
   for (j = 0; j < n; j++)
      y[i] += A[i][j]* x[j];
}
```

Matrix Matrix Multiplication: Tiling

- https://youtu.be/G92BCtfTwOE
- A.k.a tiling
- E.g., matrix-matrix multiplication
- https://youtu.be/6udqe8CkmwA?t=127

We don't go on compute this cell (1,5)

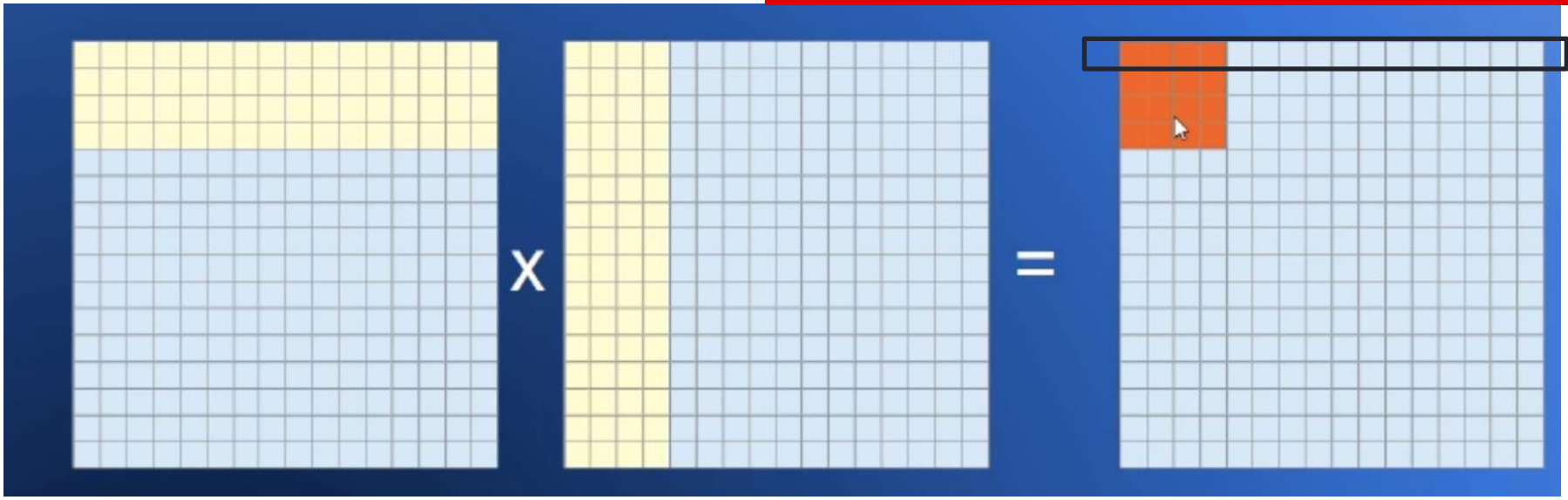


Matrix Matrix Multiplication: AxB

- Assume
  - the cache can hold t+1 rows/columns
    - Usually the cache is << # of rows/columns
  - Matrix B is **stored by columns**

No tiling, when processing the first resulting row
- Access A row 1
- Access n B columns

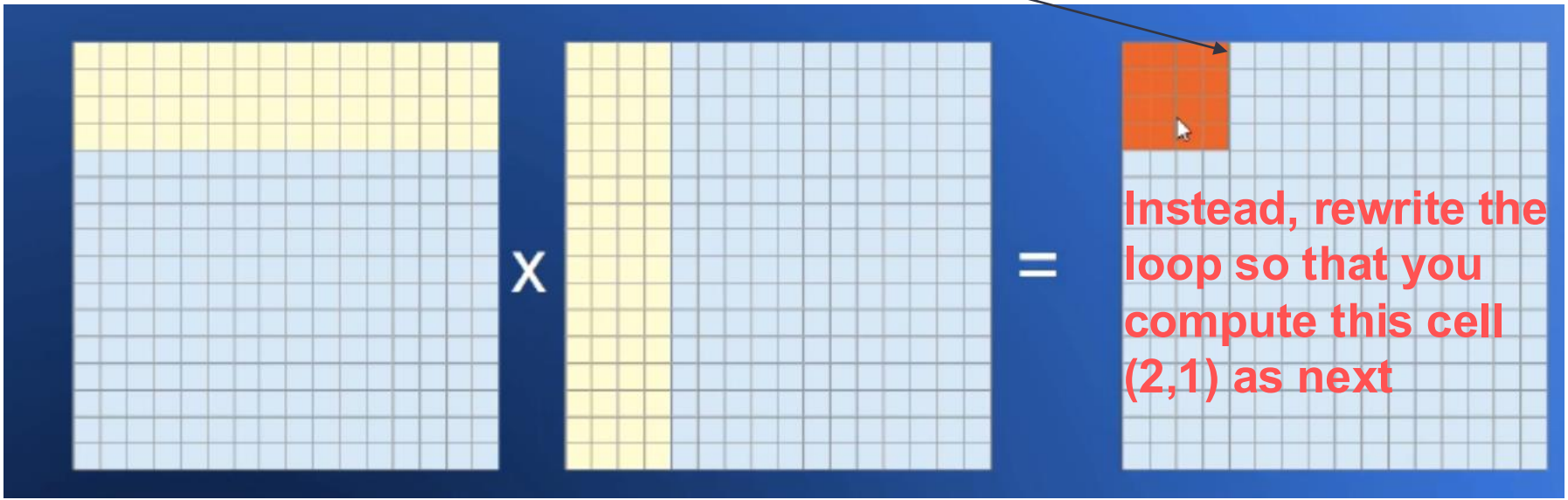
A total of n resulting rows:
= n (1 + n) misses



Matrix Matrix Multiplication: AxB

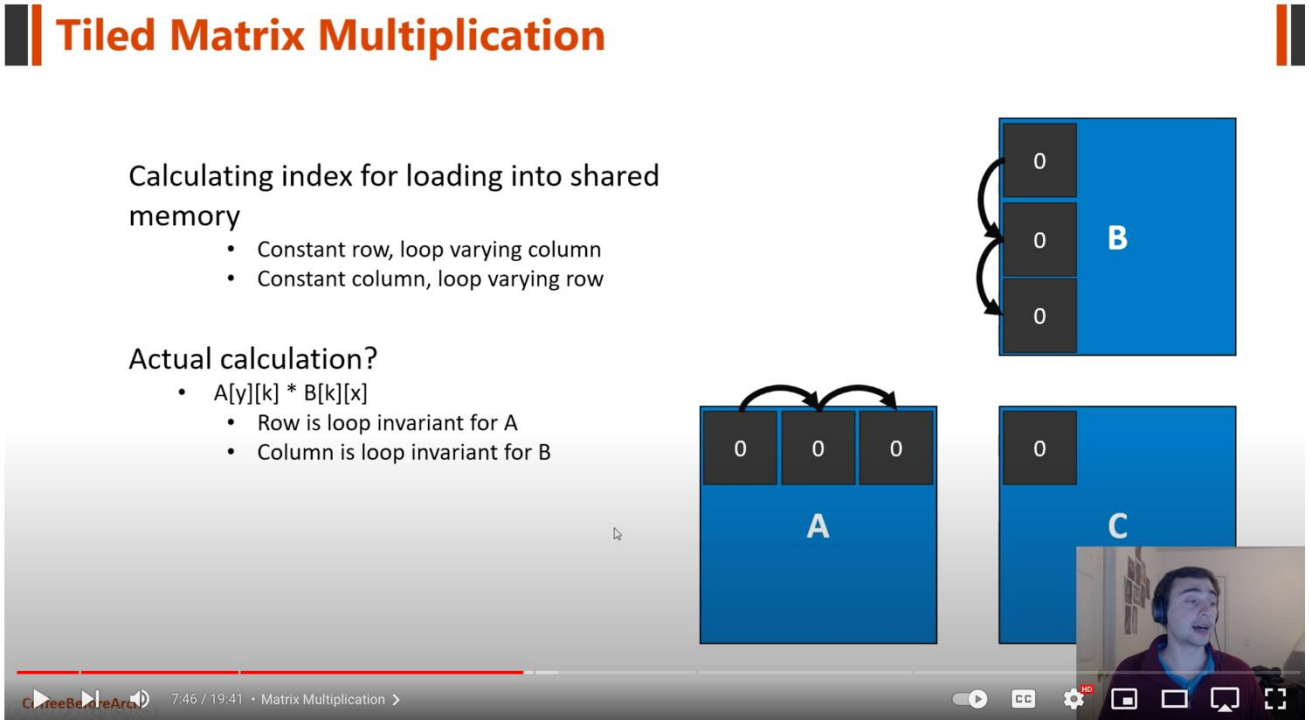
- 4x4 1st tile:
- Read 1st A row: 1 miss
- Read t B columns: t misses
- Read 2nd A row: 1 misses
- Read t B columns: 0 (cache-hit)
- A: t misses; B: t misses; Sub-Total: 2t misses

Total $(n/t)^2$ tiles:
= n^2/t^2 tites * 2t misses
= $2n^2/t$ misses
< n^2 misses //t>2



Actual tiling is more complicated

- Cacheline is often << 1 row
  - 16-256 bytes
- Cache size is often < a big matrix
  - L1 cache 1MB
    = 1 mio bytes
    = 250K floats
    = 500 x 500 matrix

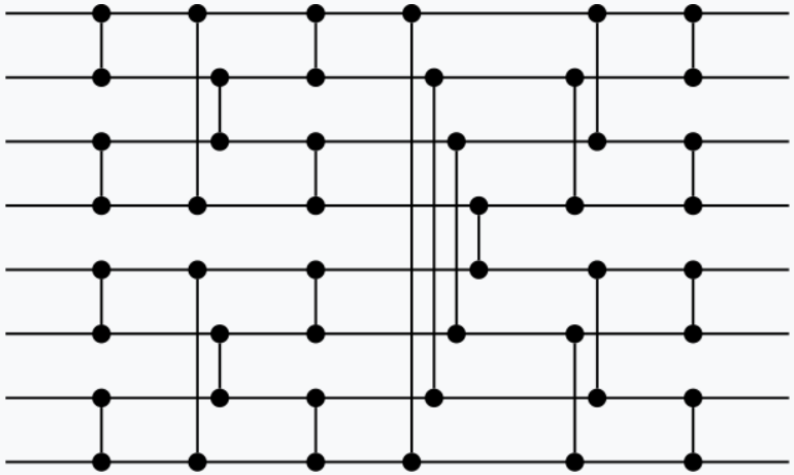


https://youtu.be/3xfyiWhtvZw

Parallel Sort

- Algorithm vs. Parallelization
- Some quick algorithms are not so amenable to parallelization
- Most parallelable sorting today: bitonic sort (sorting network)

Bitonic sorter

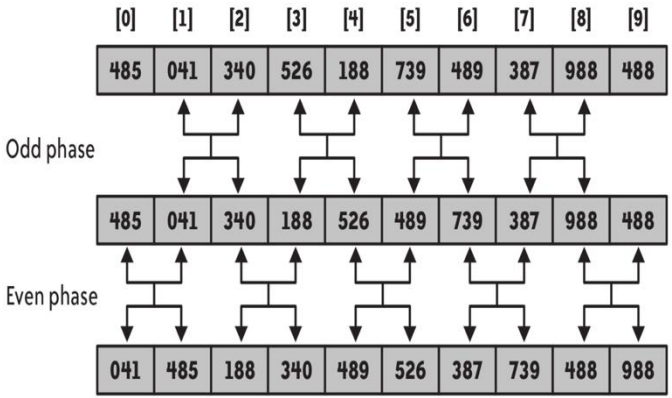


Bitonic sort network with eight inputs.

Class	Sorting algorithm
Data structure	Array
Worst-case performance	$O(\log^2(n))$ parallel time
Best-case performance	$O(\log^2(n))$ parallel time
Average performance	$O(\log^2(n))$ parallel time
Worst-case space complexity	$O(n \log^2(n))$ non-parallel time

Odd-even sort

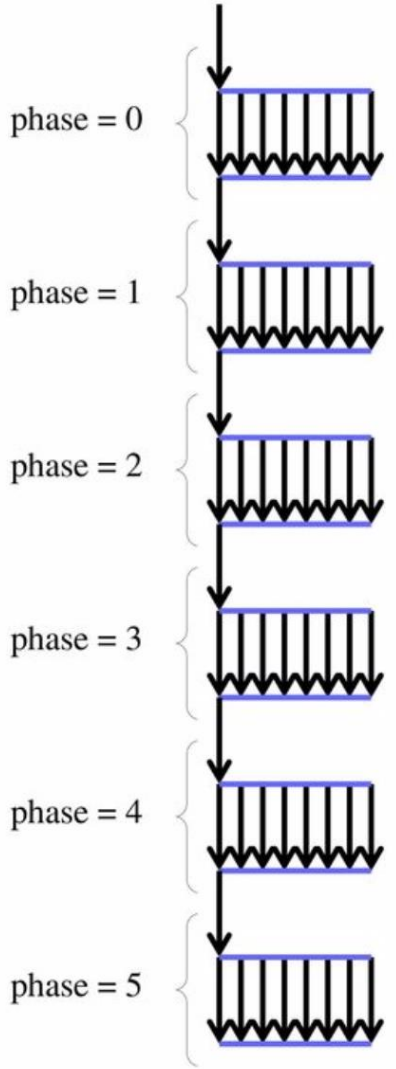
- Worst case: $O(n^2)$
- Very parallelizable
- How to implement it in OpenMP?



http://www-5.unipv.it/mferretti/cdol/cii/Multiprocessor/Programmazione%20Parallela/05%20esercizi-openmp-2.pdf
https://www.cpp.edu/~gsyoung/CS370/14Sp/parallel_sorting_kla%20Danny.pdf

- Pool of threads is being created/destroyed at each **omp parallel for** region => overhead

```
for (phase = 0; phase < n; phase++) {
      if ( phase % 2 == 0 ) {
#pragma omp parallel for default(none) shared(v,n) private(i)
            for (i=0; i<n-1; i+=2) {
                  if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );
            }
      } else {
#pragma omp parallel for default(none) shared(v,n) private(i)
            for (i=1; i<n-1; i+=2) {
                  if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );
            }
      }
}
```



```
for (phase = 0; phase < n; phase++) {
      if ( phase % 2 == 0 ) {
#pragma omp parallel for default(none) shared(v,n) private(i)
            for (i=0; i<n-1; i+=2) {
                  if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );
            }
      } else {
#pragma omp parallel for default(none) shared(v,n) private(i)
            for (i=1; i<n-1; i+=2) {
                  if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );
            }
      }
}
```

Use smarter!

```
#pragma omp parallel default(none) shared(v,n) private(i,phase)
for (phase = 0; phase < n; phase++) {
      if ( phase % 2 == 0 ) {
#pragma omp for
            for (i=0; i<n-1; i+=2) {
                  if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );
            }
      } else {
#pragma omp for
            for (i=1; i<n-1; i+=2 ) {
                  if (v[i] > v[i+1]) swap( &v[i], &v[i+1] );
            }
      }
}
```

Fork the threads

#pragma omp for uses the threads from the pool created with #pragma omp parallel

References

- https://computing.llnl.gov/tutorials/openMP/
- https://www.openmp.org/wp-content/uploads/omp-hands-on-SC08.pdf
- https://classroom.udacity.com/courses/ud281/
- https://gkaracha.github.io/papers/floyd-warshall.pdf
- http://parallelcomp.uw.hu